非线性系统

混沌系统

混沌系统（Chaotic System） 是指一个**确定的（deterministic）**、**非线性的（non-linear）** 动力学系统，其行为表现出一种极其复杂、不可预测、类似随机的现象。

> 这里的核心矛盾在于“确定的”和“不可预测的”：系统未来的所有状态都由其初始状态和固定的演化方程完全决定，没有任何随机因素介入。但由于系统内在的特性，即使初始状态有微乎其微的差异，也会导致未来的状态产生天壤之别，从而使其长期行为无法预测。

混沌时滞神经网络

考虑以下混沌时滞神经网络模型：

$$\dot{x}(t) = Ax(t) + Bf(x(t)) + \bar{B}g(x(t-\tau)) + J \tag{1}$$

其中$x(t) \in R^n$是神经元状态向量，$B, \bar{B}$ 是连接加权矩阵，J表示输入常量，τ是时滞，$f(\cdot) \in R^n, g(\cdot) \in R^n$ 是满足Lipschitz条件的激活函数，即存在Lipschitz常数L_f, L_g使得下式成立：

$$\begin{aligned} ||f(x_1) - f(x_2)|| &\leq L_f||x_1 - x_2||, \forall x_1, x_2 \in R^n \\ ||g(x_1) - g(x_2)|| &\leq L_g||x_1 - x_2||, \forall x_1, x_2 \in R^n \end{aligned}$$

> **激活函数：**
> 一个**非线性**函数。这是模仿生物神经元“要么不激活，要么就完全激活”的特性。正是这个非线性，才让系统能产生简单的线性系统无法产生的复杂行为（比如混沌）。

> **Lipschitz 条件**：
> 它限制了激活函数f和g的变化剧烈程度。L_f和L_g是两个正常数，叫做Lipschitz常数。
>
> 这个条件保证了函数的斜率不会是无限大。也就是说，当你输入改变一点点时，函数的输出也只会相应地改变一点点，变化量不会超过输入变化量的L_f倍。
>
> 在控制理论和动力系统分析中，这是一个非常重要的“良性”条件。满足这个条件的系统，其解的存在性和唯一性通常能够得到保证。也就是说，它确保了这个数学模型是“行为良好”、“可以分析”的，不会出现一些无解或解会瞬间“爆炸”到无穷大的病态情况。做控制器设计或者混沌同步分析时，这个条件是许多数学证明的基础。

> Lipschitz 条件保证了每一次放大的倍数是有限的、温和的。但是，混沌系统的反馈和迭代特性，让这个温和的放大被执行了成千上万次，最终累积成了天壤之别。

上文中系统(1)称作驱动系统，与其对应的响应系统如下：

$$\dot{\hat{x}}(t) = A\hat{x}(t) + Bf(\hat{x}(t)) + Bg(\hat{x}(t-\tau)) + u(t) + J \tag{2}$$

其中$u(t)$表示控制输入。定义$e(\cdot) = \hat{x}(t) - x(t)$,我们可以得到以下的误差系统：

$$\dot{e}(t) = Ae(t) + B(f(\hat{x}(t)) - f(x(t))) + \bar{B}(g(\hat{x}(t-\tau)) - g(x(t-\tau))) + u(t) \tag{3}$$

> **驱动系统**：在控制理论中常被称为**主系统 (Master System)** 或**参考模型 (Reference Model)**。它是一个**自主运行 (autonomous)** 的动力学系统，其演化行为完全由自身的内在动力学方程和参数决定，不受外部控制信号的干预。
>
> **响应系统**：相应地被称为**从系统 (Slave System)** 或**受控系统 (Controlled System)**。它是一个**非自主运行 (non-autonomous)** 的动力学系统，其设计目标是**追踪或同步 (track or synchronize)** 到驱动系统的状态。
>
> **误差系统**是一个派生出的动力学系统，它描述了**同步误差 (synchronization error)** 随时间的演化规律。**同步误差**被定义为响应系统状态与驱动系统状态之间的向量差。

我们的目的是设计具有如下形式的控制器：

$$u(t) = Ke(t) \tag{4}$$

使得误差系统状态趋于零，其中K是待求的控制增益。

定义一：渐进同步

如果同步误差$e(t)$满足以下条件：

$$\lim_{t\to\infty} e(t) = 0$$

则驱动系统(1)和响应系统(2)是渐进同步的。

引理一（基于杨氏不等式）

对于具有适当维数的任意实矩阵X和Y,存在一个常数$\varepsilon > 0$使得

$$XY^T + YX^T \leq \frac{1}{\varepsilon}XX^T + \varepsilon YY^T$$

定理一

若存在矩阵$P > 0, Q > 0, M$以及标量ε使得以下不等式成立

$$\begin{bmatrix} He\{PA+M\}+Q+\frac{1}{\varepsilon}L_f^2 I & 0 & PB & P\bar{B} \\ * & -Q+\frac{1}{\varepsilon}L_g^2 I & 0 & 0 \\ * & * & -\varepsilon I & 0 \\ * & * & * & -\varepsilon I \end{bmatrix} < 0$$

则系统 (1)和(2) 渐近同步，此时可通过$K = P^{-1}M$求得控制增益。

证明:

构造如下李雅普诺夫泛函:

$$V(e(t)) = e^T(t)Pe(t) + \int_{t-\tau}^{t} e^T(s)Qe(s)ds$$

求导得:

$$\dot{V}\left(e\left(t\right)\right) = 2e^T\left(t\right)P\dot{e}\left(t\right) + e^T\left(t\right)Qe\left(t\right) - e^T\left(t-\tau\right)Qe\left(t-\tau\right)$$

将(3)式代入得:

$$\dot{V}\left(e\left(t\right)\right) = 2e^T\left(t\right)P\Big(Ae\left(t\right) + B\left(f\left(\hat{x}\left(t\right)\right) - f\left(x\left(t\right)\right)\right) + \bar{B}\left(g\left(\hat{x}\left(t-\tau\right)\right) - g\left(x\left(t-\tau\right)\right)\right) + u\left(t\right)\Big) + e^T\left(t\right)Qe\left(t\right) - e^T\left(t-\tau\right)Qe\left(t-\tau\right)$$

将(4)式代入并化简得:

$$\dot{V}\left(e\left(t\right)\right) = e^T\left(t\right)\left(He\left\{PA+PK\right\}+Q\right)e\left(t\right) + 2e^T\left(t\right)PB\left(f\left(\hat{x}\left(t\right)\right) - f\left(x\left(t\right)\right)\right) + 2e^T\left(t\right)P\bar{B}\left(g\left(\hat{x}\left(t-\tau\right)\right) - g\left(x\left(t-\tau\right)\right)\right) - e^T\left(t-\tau\right)Qe\left(t-\tau\right)$$

结合Lipschitz条件与**引理一**:

$$\begin{aligned} 2e^T\left(t\right)PB\left(f\left(\hat{x}\left(t\right)\right) - f\left(x\left(t\right)\right)\right) &= \left(f\left(\hat{x}\left(t\right)\right) - f\left(x\left(t\right)\right)\right)^T\left(e^T\left(t\right)PB\right)^T + e^T\left(t\right)PB\Big(\left(f\left(\hat{x}\left(t\right)\right) - f\left(x\left(t\right)\right)\right)^T\Big)^T \\ &\leq \frac{1}{\varepsilon}\left(f\left(\hat{x}\left(t\right)\right) - f\left(x\left(t\right)\right)\right)^T\left(f\left(\hat{x}\left(t\right)\right) - f\left(x\left(t\right)\right)\right) + \varepsilon e^T\left(t\right)PBB^TPe\left(t\right) \\ &\leq \frac{1}{\varepsilon}L_f^2(\dot{x}\left(t\right) - x\left(t\right))^T\left(\dot{x}\left(t\right) - x\left(t\right)\right) + \varepsilon e^T\left(t\right)PBB^TPe\left(t\right) \\ &= \frac{1}{\varepsilon}L_f^2 e^T\left(t\right)e\left(t\right) + \varepsilon e^T\left(t\right)PBB^TPe\left(t\right) \end{aligned}$$

同理得:

$$2e^T\left(t\right)P\bar{B}\left(g\left(\hat{x}\left(t-\tau\right)\right) - g\left(x\left(t-\tau\right)\right)\right) \leq \frac{1}{\varepsilon}L_g^2 e^T\left(t-\tau\right)e\left(t-\tau\right) + \varepsilon e^T\left(t\right)P\bar{B}\bar{B}^TPe\left(t\right)$$

代入(5)式得:

$$\begin{aligned} &\dot{V}\left(e\left(t\right)\right) = e^T\left(t\right)\left(He\left\{PA+PK\right\}+Q\right)e\left(t\right) + 2e^T\left(t\right)PB\left(f\left(\hat{x}\left(t\right)\right) - f\left(x\left(t\right)\right)\right) + 2e^T\left(t\right)P\bar{B}\left(g\left(\hat{x}\left(t-\tau\right)\right) - g\left(x\left(t-\tau\right)\right)\right) - e^T\left(t-\tau\right)Qe\left(t-\tau\right) \\ &\leq e^T\left(t\right)\left(He\left\{PA+PK\right\}+Q\right)e\left(t\right) + \frac{1}{\varepsilon}L_f^2 e^T\left(t\right)e\left(t\right) + \varepsilon e^T\left(t\right)PBB^TPe\left(t\right) + \frac{1}{\varepsilon}L_g^2 e^T\left(t-\tau\right)e\left(t-\tau\right) + \varepsilon e^T\left(t\right)P\bar{B}\bar{B}^TPe\left(t\right) - e^T\left(t-\tau\right)Qe\left(t-\tau\right) \\ &= \begin{bmatrix} e\left(t\right) \\ e\left(t-\tau\right) \end{bmatrix}^T \begin{bmatrix} He\left\{PA+PK\right\}+Q+\frac{1}{\varepsilon}L_f^2+\varepsilon PBB^TP+\varepsilon P\bar{B}\bar{B}^TP & 0 \\ * & -Q+\frac{1}{\varepsilon}L_g^2 \end{bmatrix} \begin{bmatrix} e\left(t\right) \\ e\left(t-\tau\right) \end{bmatrix} \end{aligned}$$

若

$$\begin{bmatrix} He\left\{PA+PK\right\}+Q+\frac{1}{\varepsilon}L_f^2+\varepsilon PBB^TP+\varepsilon P\bar{B}\bar{B}^TP & 0 \\ * & -Q+\frac{1}{\varepsilon}L_g^2 \end{bmatrix} < 0 \tag{6}$$

则系统(3)渐进稳定，所以系统(1)(2)渐进同步。令$PK = M$，运用$Schur$补定理，(6)式等价于:

$$\begin{bmatrix} He\{PA+M\}+Q+\frac{1}{\varepsilon}L_f^2 I & 0 & PB & P\bar{B} \\ * & -Q+\frac{1}{\varepsilon}L_g^2 I & 0 & 0 \\ * & * & -\varepsilon I & 0 \\ * & * & * & -\varepsilon I \end{bmatrix} < 0$$

此时控制增益矩阵$K = P^{-1}M$。

仿真示例

考虑如下$Hopfiled$神经网络:

$$\begin{bmatrix} \dot{x}_1(t) \\ \dot{x}_2(t) \end{bmatrix} = \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix} + \begin{bmatrix} 2 & -0.1 \\ -5 & 2 \end{bmatrix} \begin{bmatrix} \tan\mathrm{h}(x_1(t)) \\ \tan\mathrm{h}(x_2(t)) \end{bmatrix} + \begin{bmatrix} -1.5 & -0.1 \\ -0.2 & -1.5 \end{bmatrix} \begin{bmatrix} \tan\mathrm{h}(x_1(t-1)) \\ \tan\mathrm{h}(x_2(t-1)) \end{bmatrix}$$

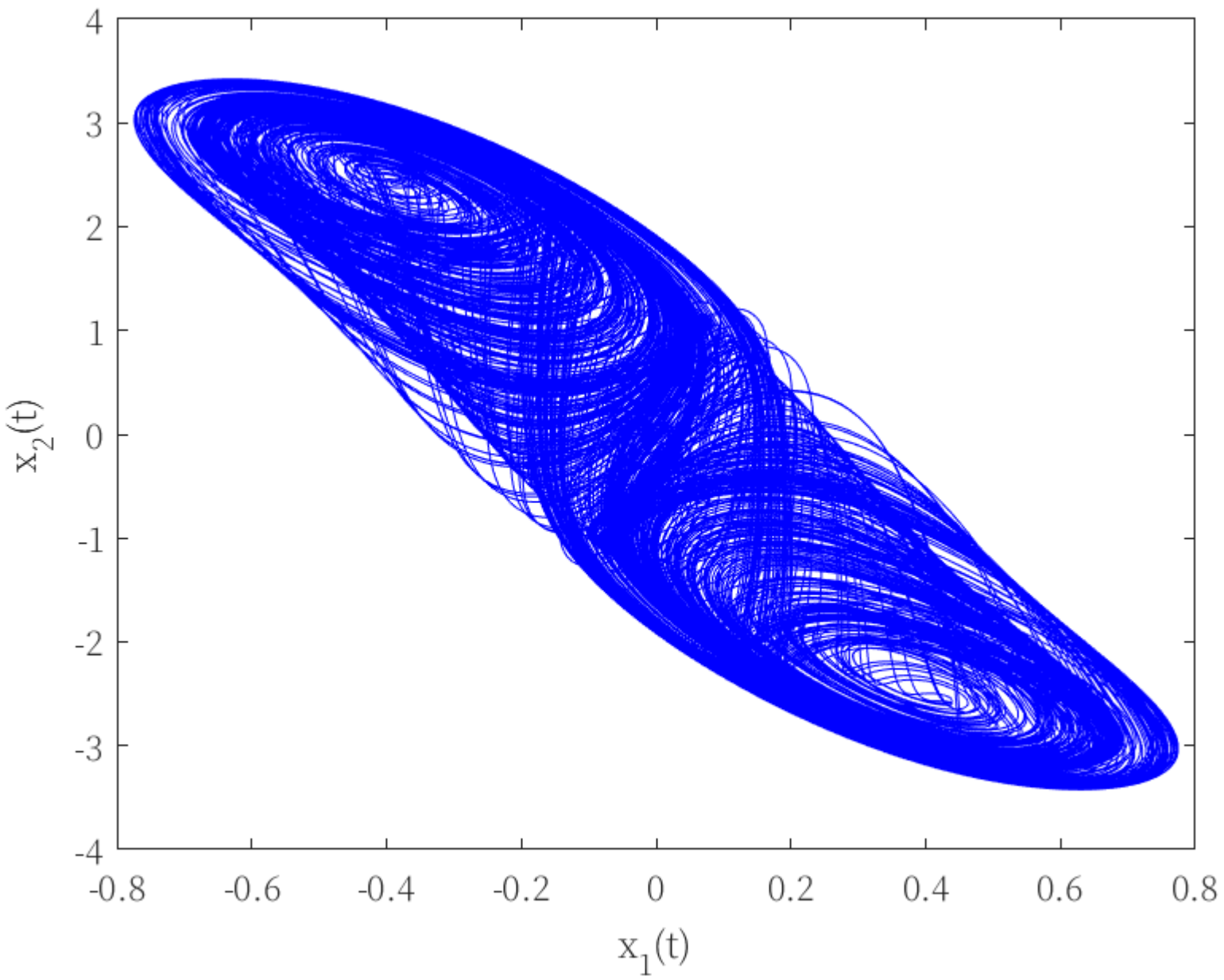
其中驱动系统与响应系统的初值分别为:

$$\begin{bmatrix} x_1(s) \\ x_2(s) \end{bmatrix} = \begin{bmatrix} 1.7 \\ 2.5 \end{bmatrix}, \begin{bmatrix} \hat{x}_1(s) \\ \hat{x}_2(s) \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, s \in [-1, 0]$$

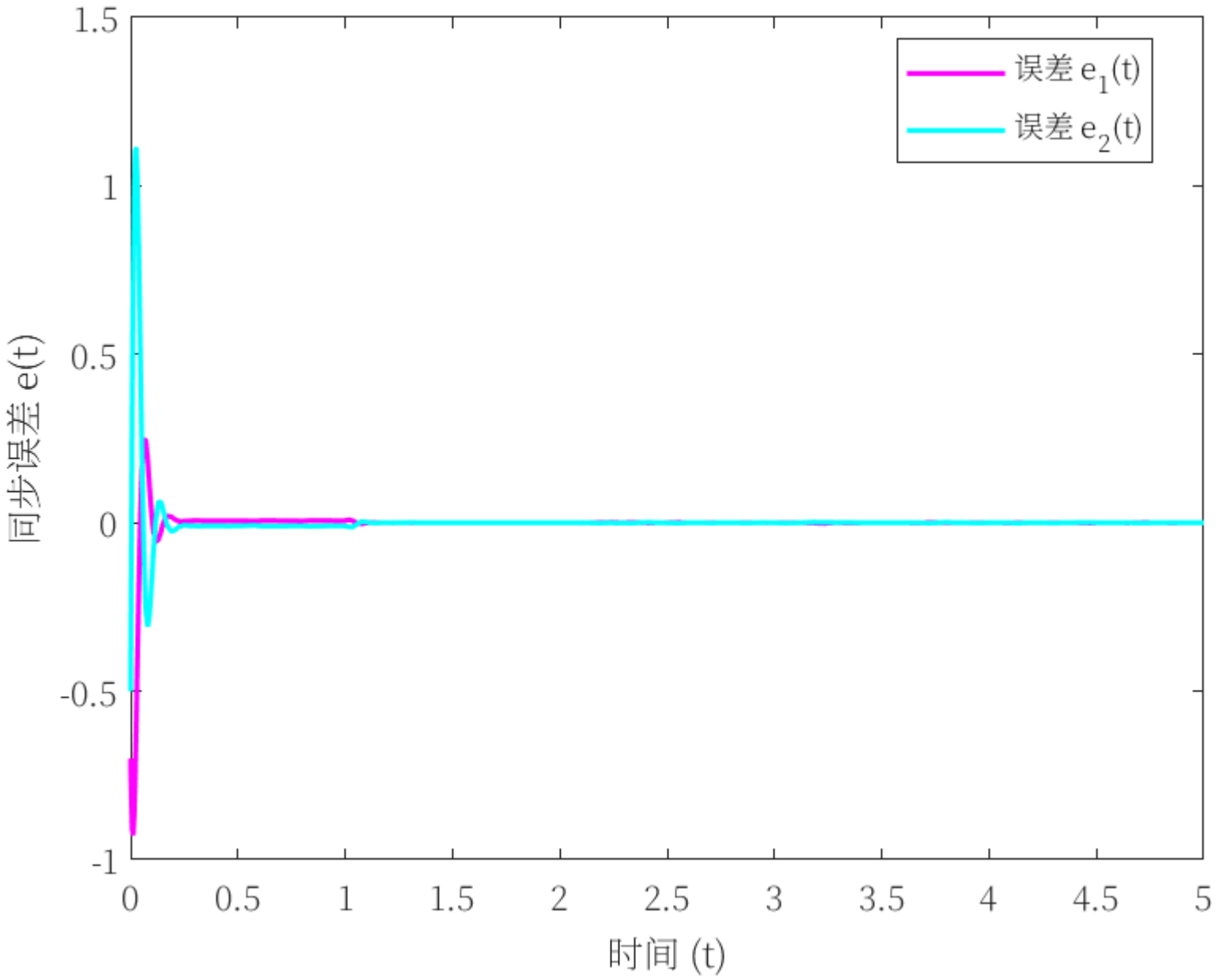
解得:

$$K = \begin{bmatrix} 32.8175 & 47.7361 \\ -133.1490 & -79.0097 \end{bmatrix}$$

得到如图所示的**驱动系统的相平面轨迹图**:



误差系统的状态轨迹如下所示:



代码如下:

```MATLAB
clc
clear
A = [-1, 0; 0, -1];
B = [2, -0.1; -5, 2];
B_hat = [-1.5, -0.1; -0.2, -1.5];
Lf = 1;
Lg = 1;

setlmis([])
P = lmivar(1, [2, 1]);
Q = lmivar(1, [2, 1]);
M = lmivar(2, [2, 2]);

lmiterm([1, 1, 1, P], 1, A, 's')
lmiterm([1, 1, 1, M], 1, 1, 's')
lmiterm([1, 1, 1, Q], 1, 1)
lmiterm([1, 1, 1, 0], Lf*eye(2))
lmiterm([1, 2, 2, Q], -1, 1)
lmiterm([1, 2, 2, 0], Lg*eye(2))
lmiterm([1, 1, 3, P], 1, B)
lmiterm([1, 3, 3, 0], -eye(2))
lmiterm([1, 1, 4, P], 1, B_hat)
lmiterm([1, 4, 4, 0], -eye(2))
lmiterm([2, 1, 1, P], -1, 1)
lmiterm([3, 1, 1, Q], -1, 1)

lmis = getlmis;
[tmin, xfeas] = feasp(lmis);

if tmin < 0
    fprintf("LMI求解成功! \n")
    P = dec2mat(lmis, xfeas, P)
    M = dec2mat(lmis, xfeas, M)
    K = (P^-1) * M
    tmin
else
    fprintf("LMI求解失败, 问题无解。\n")
end
```

MATLAB

```matlab
%%%
```

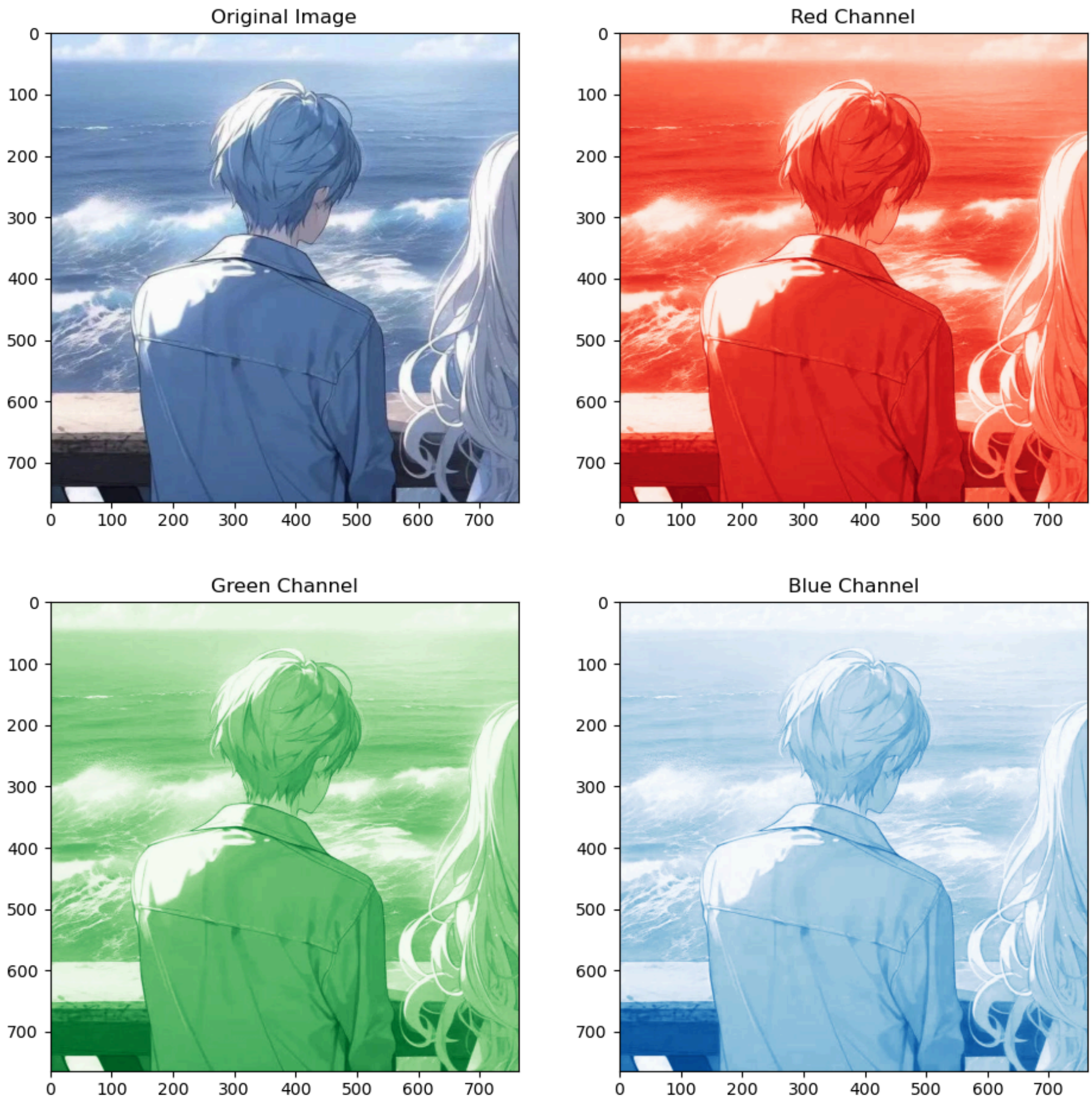
MATLAB

```matlab
% 误差系统的轨迹
% (通过求解 驱动-误差 耦合系统)

clear;
clc;

%% 1. 定义系统参数和控制器增益
A = [-1, 0; 0, -1];
B = [2, -0.1; -5, 2];
B_hat = [-1.5, -0.1; -0.2, -1.5];
K = [32.8175, 47.7361; ...
    -133.1490, -79.0097];

%% 2. 定义 驱动-误差 耦合的DDE系统函数
% Y = [x; e], Z = [x_lag; e_lag]
ddefun = @(t, Y, Z) drive_error_dde(t, Y, Z, A, B, B_hat, K);

%% 3. 设置求解参数
lags = 1; % 时滞 tau = 1

% 设定驱动系统的历史函数 (初始值)
history_x = [1.7; 2.5];
% 设定响应系统的历史函数 (初始值)
history_x_hat = [1; 2];
% 计算初始误差
history_e = history_x_hat - history_x;

% 合并成4维历史向量 [x_history; e_history]
history = [history_x; history_e];

% 设定求解时间区间
tspan = [0, 5];

%% 4. 调用 dde23 求解
fprintf('正在求解 驱动-误差 耦合系统...\n');
sol = dde23(ddefun, lags, history, tspan);
fprintf('求解完成。\n');

%% 5. 提取并绘制误差结果
t = sol.x;
Y = sol.y;

% 直接从结果中分离出误差系统的状态 e(t)
e = Y(3:4, :);

% 绘制同步误差的状态轨迹
figure;
plot(t, e(1, :), 'm-', 'LineWidth', 1.5);
hold on;
plot(t, e(2, :), 'c-', 'LineWidth', 1.5);
grid on;
title('同步误差 e(t) = \hat{x}(t) - x(t) 的状态轨迹');
xlabel('时间 (t)');
ylabel('同步误差 e(t)');
legend('误差 e_1(t)', '误差 e_2(t)', 'Location', 'best');
hold off;

%% DDE 系统定义 (作为脚本的局部函数)
function dydt = drive_error_dde(t, Y, Z, A, B, B_hat, K)
% 从4维向量中分离出状态
x = Y(1:2);
e = Y(3:4);

% 分离出时滞状态
x_lag = Z(1:2);
e_lag = Z(3:4);

% 计算驱动系统的导数
dxdt = A * x + B * tanh(x) + B_hat * tanh(x_lag);

% 根据需要，在函数内部重构响应系统的状态
x_hat = x + e;
x_hat_lag = x_lag + e_lag;
```

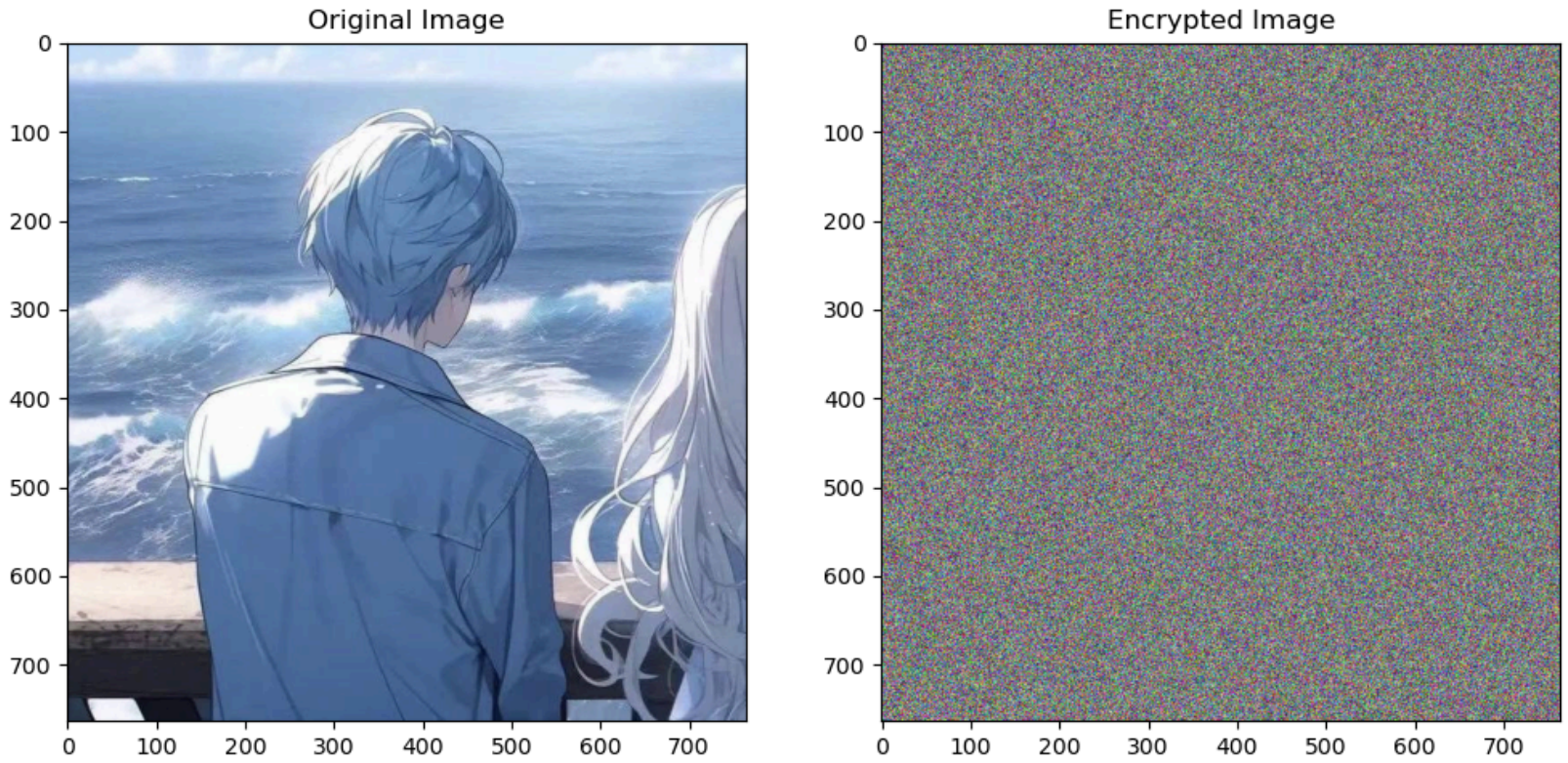
```
% 计算误差系统的导数
dedt = A * e + B * (tanh(x_hat) - tanh(x)) + B_hat * (tanh(x_hat_lag) - tanh(x_lag)) + K * e;

% 将导数组合成4维输出向量
dydt = [dxdt; dedt];
end
```

图像加密

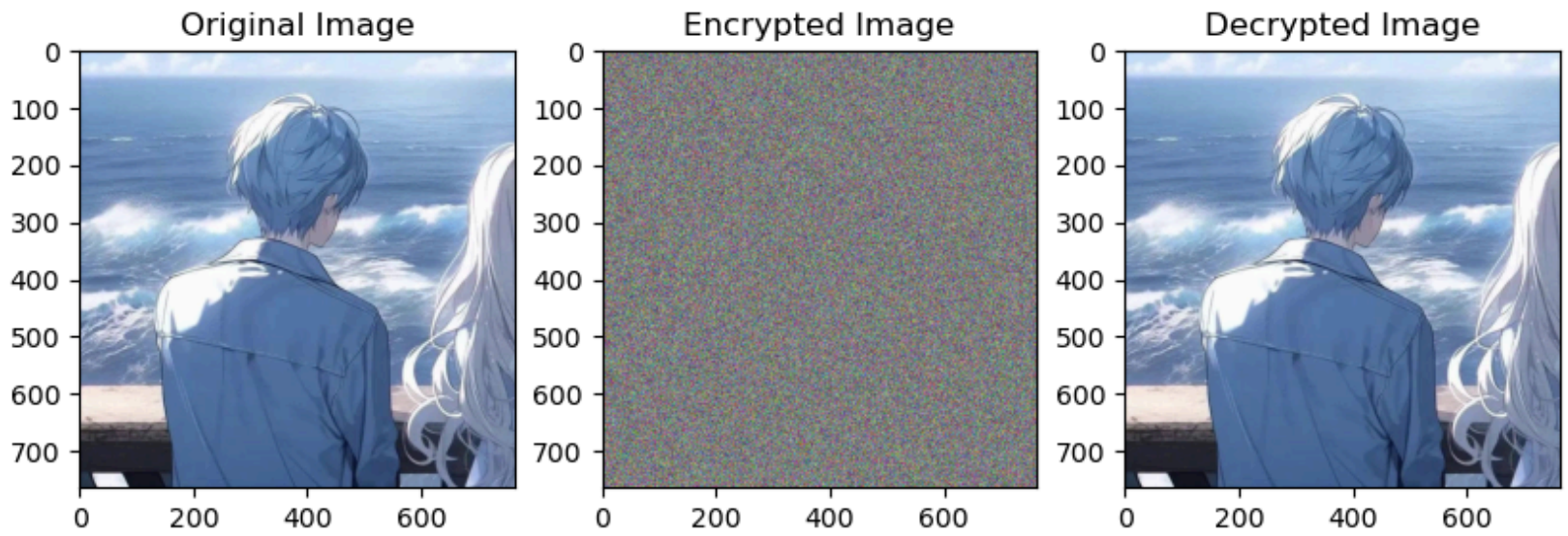
以**微信头像**为例，其RGB三个通道图分别如下所示：



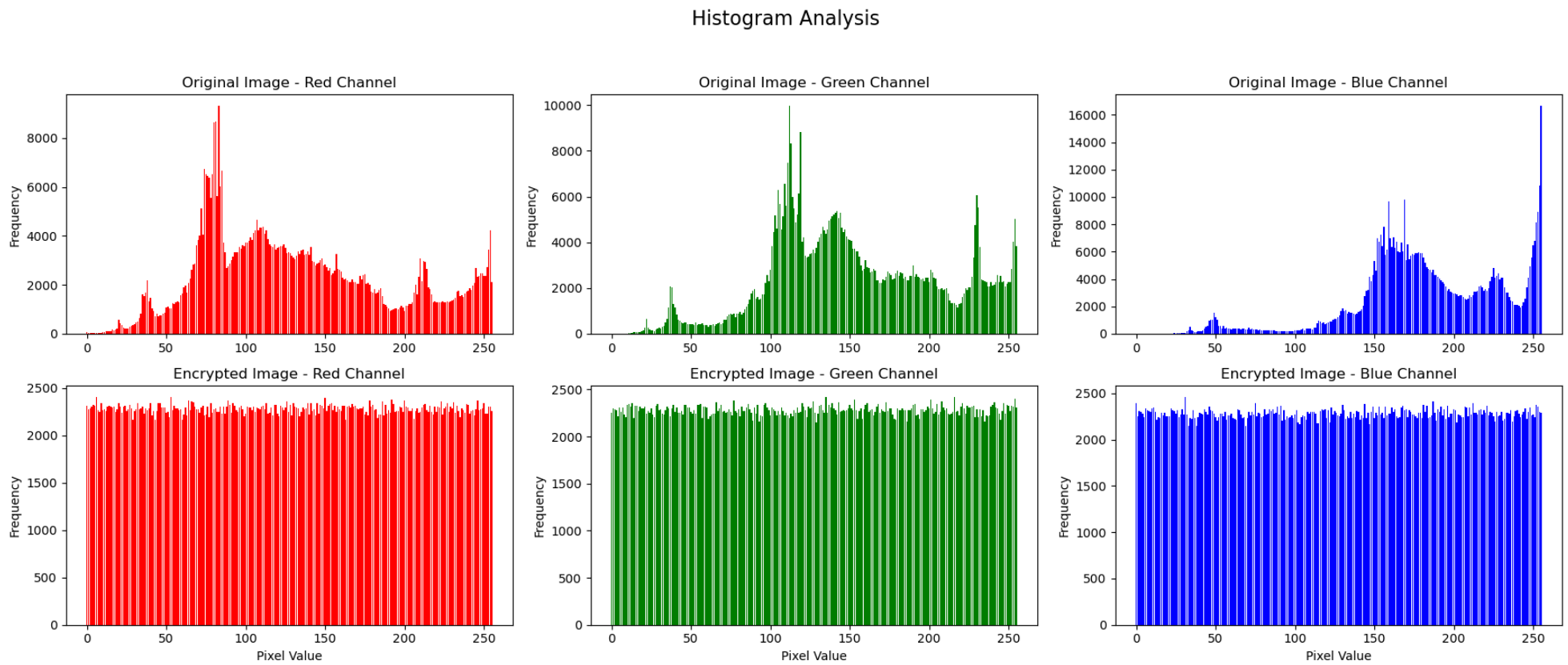
加密后



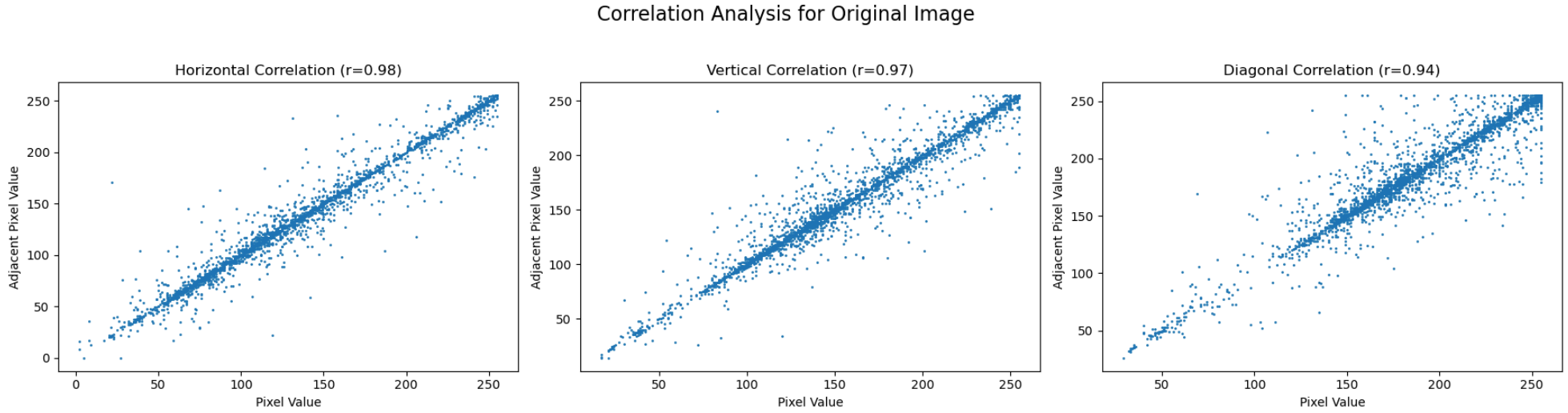
解密后:

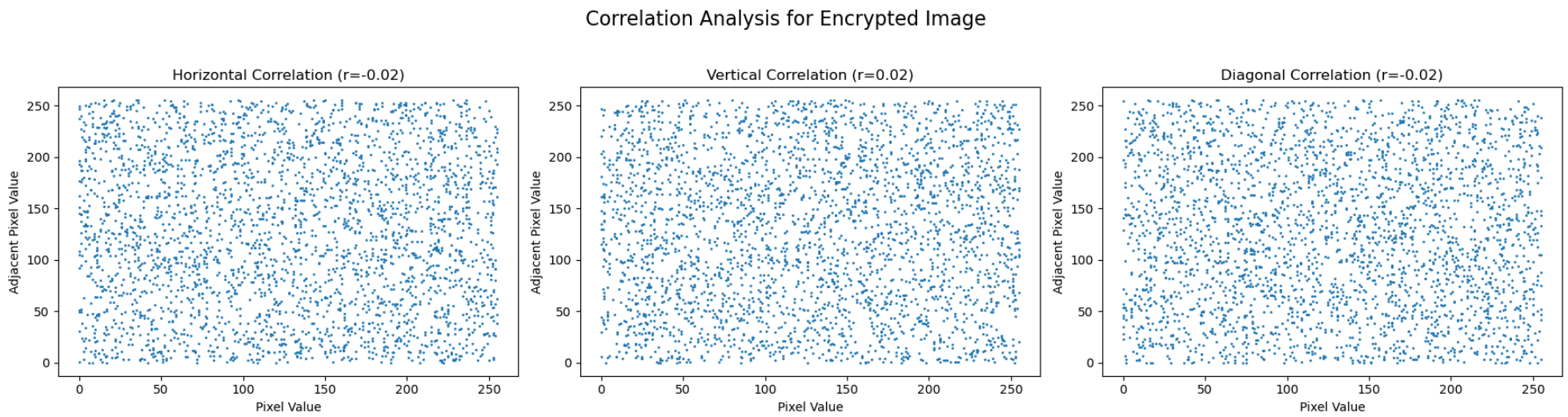


绘制原图与加密图的灰度直方图:



绘制原图与加密图的相关性图





可以看出加密后的图像没有明显的统计信息。

附录：图像加密的Python代码

PYTHON

```python
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

image_path = "p1.png"

original_image = Image.open(image_path)
image_array = np.array(original_image)

# 分离R, G, B三个颜色通道
r_channel = image_array[:, :, 0]
g_channel = image_array[:, :, 1]
b_channel = image_array[:, :, 2]

fig, axes = plt.subplots(2, 2, figsize=(10, 10))
fig.tight_layout(pad=3.0)

axes[0, 0].imshow(original_image)
axes[0, 0].set_title("Original Image")
# axes[0, 0].axis("off")

axes[0, 1].imshow(r_channel, cmap="Reds_r")
axes[0, 1].set_title("Red Channel")
# axes[0, 1].axis("off")

axes[1, 0].imshow(g_channel, cmap="Greens_r")
axes[1, 0].set_title("Green Channel")
# axes[1, 0].axis("off")

axes[1, 1].imshow(b_channel, cmap="Blues_r")
axes[1, 1].set_title("Blue Channel")
# axes[1, 1].axis("off")

output_filename = "p1_1.png"
plt.savefig(output_filename)

plt.show()
```

PYTHON

```python
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image


def generate_chaotic_keys(height, width):
    A = np.array([[-1, 0], [0, -1.5]])
    B = np.array([[2, -0.1], [-5, 2]])
    B_bar = np.array([[-1.5, -0.1], [-0.2, -1]])
    tau = 1.0

    x0 = np.array([1.7, 2.5])
    dt = 0.01
    sim_time = 10

    total_pixels = height * width
    required_key_points = total_pixels * 3

    burn_in_steps = int(sim_time / dt)
    history_steps = int(tau / dt)
    total_steps = burn_in_steps + required_key_points

    history = np.zeros((total_steps + 1, 2))
    history[0, :] = x0
    x = x0

    for i in range(total_steps):
        past_index = i - history_steps
        if past_index < 0:
            x_tau = x0
        else:
            x_tau = history[past_index, :]

        f_x = np.tanh(x)
        g_x_tau = np.tanh(x_tau)

        x_dot = A @ x + B @ f_x + B_bar @ g_x_tau

        x = x + dt * x_dot
        history[i + 1, :] = x

    chaotic_swquence = history[burn_in_steps + 1 :, :]

    norms = np.linalg.norm(chaotic_swquence, axis=1)
    keys_raw = np.mod(np.round(norms * 1e8), 256).astype(np.uint8)

    keys_r = keys_raw[0:total_pixels]
    keys_g = keys_raw[total_pixels : 2 * total_pixels]
    keys_b = keys_raw[2 * total_pixels : 3 * total_pixels]

    return keys_r, keys_g, keys_b


def encrypt_image(image_array, keys):
    height, width, _ = image_array.shape
    key_r, key_g, key_b = keys
    key_r_2d = key_r.reshape((height, width))
    key_g_2d = key_g.reshape((height, width))
    key_b_2d = key_b.reshape((height, width))

    encrypt_image = image_array.copy()

    encrypt_image[:, :, 0] = np.bitwise_xor(image_array[:, :, 0], key_r_2d)
    encrypt_image[:, :, 1] = np.bitwise_xor(image_array[:, :, 1], key_g_2d)
    encrypt_image[:, :, 2] = np.bitwise_xor(image_array[:, :, 2], key_b_2d)
    return Image.fromarray(encrypt_image)


if __name__ == "__main__":
    image_path = "p1.png"
    original_image = Image.open(image_path)
    image_array = np.array(original_image)
    height, width, _ = image_array.shape
```

```
keys = generate_chaotic_keys(height, width)
encrypted_image = encrypt_image(image_array, keys)

fig, axes = plt.subplots(1, 2, figsize=(12, 6))
axes[0].imshow(original_image)
axes[0].set_title("Original Image")

axes[1].imshow(encrypted_image)
axes[1].set_title("Encrypted Image")

output_filename = "p1_2.png"
plt.savefig(output_filename)
plt.show()

output_filename = "p1_2_2.png"
encrypted_image.save(output_filename)
```

PYTHON

```python
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image


def generate_chaotic_keys(height, width):
    A = np.array([[-1, 0], [0, -1.5]])
    B = np.array([[2, -0.1], [-5, 2]])
    B_bar = np.array([[-1.5, -0.1], [-0.2, -1]])
    tau = 1.0

    x0 = np.array([1.7, 2.5])
    dt = 0.01
    sim_time = 10

    total_pixels = height * width
    required_key_points = total_pixels * 3

    burn_in_steps = int(sim_time / dt)
    history_steps = int(tau / dt)
    total_steps = burn_in_steps + required_key_points

    history = np.zeros((total_steps + 1, 2))
    history[0, :] = x0
    x = x0

    for i in range(total_steps):
        past_index = i - history_steps
        if past_index < 0:
            x_tau = x0
        else:
            x_tau = history[past_index, :]

        f_x = np.tanh(x)
        g_x_tau = np.tanh(x_tau)

        x_dot = A @ x + B @ f_x + B_bar @ g_x_tau

        x = x + dt * x_dot
        history[i + 1, :] = x

    chaotic_swquence = history[burn_in_steps + 1 :, :]

    norms = np.linalg.norm(chaotic_swquence, axis=1)
    keys_raw = np.mod(np.round(norms * 1e8), 256).astype(np.uint8)

    keys_r = keys_raw[0:total_pixels]
    keys_g = keys_raw[total_pixels : 2 * total_pixels]
    keys_b = keys_raw[2 * total_pixels : 3 * total_pixels]

    return keys_r, keys_g, keys_b


def decrypt_image(encrypted_image, keys):
    height, width, _ = encrypted_image.shape
    key_r, key_g, key_b = keys

    key_r_2d = key_r.reshape((height, width))
    key_g_2d = key_g.reshape((height, width))
    key_b_2d = key_b.reshape((height, width))

    decrypted_array = encrypted_image.copy()

    decrypted_array[:, :, 0] = encrypted_image[:, :, 0] ^ key_r_2d
    decrypted_array[:, :, 1] = encrypted_image[:, :, 1] ^ key_g_2d
    decrypted_array[:, :, 2] = encrypted_image[:, :, 2] ^ key_b_2d
    return Image.fromarray(decrypted_array)


if __name__ == "__main__":
    original_image_path = "p1.png"
    original_image = Image.open(original_image_path)

    encrypted_image_path = "p1_2_2.png"
```

```
encrypted_image = Image.open(encrypted_image_path)

encrypted_image_array = np.array(encrypted_image)
height, width, _ = encrypted_image_array.shape
keys = generate_chaotic_keys(height, width)
decrypted_image = decrypt_image(encrypted_image_array, keys)

fig, axes = plt.subplots(1, 3, figsize=(10, 6))
axes[0].imshow(original_image)
axes[0].set_title("Original Image")

axes[1].imshow(encrypted_image)
axes[1].set_title("Encrypted Image")

axes[2].imshow(decrypted_image)
axes[2].set_title("Decrypted Image")

output_filename = "p3.png"
plt.savefig(output_filename)
plt.show()
```

PYTHON

```python
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image


def analyze_histogram(original_image_array, encrypted_image_array):
    fig, axes = plt.subplots(2, 3, figsize=(18, 8))
    fig.suptitle("Histogram Analysis", fontsize=16)

    for i, color in enumerate(("red", "green", "blue")):
        hist = np.histogram(
            original_image_array[:, :, i].ravel(), bins=256, range=[0, 256]
        )
        axes[0, i].bar(hist[1][:-1], hist[0], color=color)
        axes[0, i].set_title(f"Original Image - {color.capitalize()} Channel")
        axes[0, i].set_ylabel("Frequency")

    for i, color in enumerate(("red", "green", "blue")):
        hist = np.histogram(
            encrypted_image_array[:, :, i].ravel(), bins=256, range=[0, 256]
        )
        axes[1, i].bar(hist[1][:-1], hist[0], color=color)
        axes[1, i].set_title(f"Encrypted Image - {color.capitalize()} Channel")
        axes[1, i].set_xlabel("Pixel Value")
        axes[1, i].set_ylabel("Frequency")

    plt.tight_layout(rect=[0, 0.03, 1, 0.95])
    plt.savefig("p4.png")
    plt.show()


def analyze_correlation(image_array, title="Image"):
    num_pixels = 3000
    height, width, _ = image_array.shape
    x_coords = np.random.randint(0, height - 1, num_pixels)
    y_coords = np.random.randint(0, width - 1, num_pixels)
    fig, axes = plt.subplots(1, 3, figsize=(18, 5))
    fig.suptitle(f"Correlation Analysis for {title}", fontsize=16)

    directions = ["Horizontal", "Vertical", "Diagonal"]

    for i, direction in enumerate(directions):
        x_values = image_array[x_coords, y_coords, i % 3]
        if direction == "Horizontal":
            y_values = image_array[x_coords, y_coords + 1, i % 3]
        elif direction == "Vertical":
            y_values = image_array[x_coords + 1, y_coords, i % 3]
        else:
            y_values = image_array[x_coords + 1, y_coords + 1, i % 3]

        corr = np.corrcoef(x_values, y_values)[0, 1]
        axes[i].scatter(x_values, y_values, s=1)
        axes[i].set_title(f"{direction} Correlation (r={corr:.2f})")
        axes[i].set_xlabel("Pixel Value")
        axes[i].set_ylabel("Adjacent Pixel Value")

    plt.tight_layout(rect=[0, 0.03, 1, 0.95])
    plt.savefig(f"p5_{title.lower().replace(' ', '_')}.png")
    plt.show()


if __name__ == "__main__":
    original_image = Image.open("p1.png")
    original_image_array = np.array(original_image)

    encrypted_image = Image.open("p1_2_2.png")
    encrypted_image_array = np.array(encrypted_image)
    analyze_histogram(original_image_array, encrypted_image_array)
    analyze_correlation(original_image_array, "Original Image")
    analyze_correlation(encrypted_image_array, "Encrypted Image")
```